

SemanticEmoji: Multi-lingual Neural Emoji Search

Course: DA6401 Introduction to Deep Learning

Author: Sohang Chopra (DA25M622)

Date: April 29, 2026

Abstract

Traditional emoji search engines rely heavily on keyword-based matching, failing to capture semantic nuances or synonyms. This project introduces **SemanticEmoji**, a deep learning-based approach to emoji retrieval that maps natural language descriptions to emoji classes using neural embeddings. By leveraging a pre-trained sentence-transformer (**all-mpnet-base-v2**) as a feature extractor and a custom Multi-Layer Perceptron (MLP) classifier, the system achieves a Top-5 accuracy of 92.46%. The model is designed for efficiency, enabling offline deployment on CPU-constrained devices like mobile phones or edge hardware, thereby eliminating the need for high-latency internet-based LLM queries.

1. Introduction

Emojis have become an integral part of modern digital communication. However, finding the right emoji often proves frustrating due to the limitations of current keyboard search implementations, which typically use exact substring matching. For instance, searching for "joy" might not return the "laughing face" emoji if the metadata only contains the word "happy." This project aims to solve this by treating emoji search as a semantic classification problem.

2. Background and State-of-the-art

State-of-the-art emoji retrieval often involves large-scale Language Models (LLMs) which are computationally expensive. On the other end of the spectrum, keyword-based systems are lightweight but lack intelligence. This project utilizes "Sentence Embeddings," which represent sentences in a high-dimensional vector space where semantically similar meanings are geographically close, providing a middle ground between efficiency and performance.

3. Problem Statement

The objective is to design a multi-class classification system that maps an arbitrary natural language string (e.g., "feeling cold") to its most relevant emoji (e.g., 🥶). The system must:

- Support semantic understanding beyond keywords.
- Be lightweight enough for CPU-only inference.
- Provide a ranked list (Top-5) of emojis to account for semantic overlap.

4. Dataset Description

The project uses a synthetic dataset generated via Gemini LLM to ensure high-quality descriptions and volume.

- **Classes:** 96 unique emoji categories.
- **Samples:** 50 unique natural language descriptions per emoji (4,800 total samples).
- **Structure:** A mapping of emoji characters to diverse textual descriptions including synonyms and contextual usage.

5. Solution / Methodology / Neural Net Architecture

The architecture follows a two-stage pipeline:

1. **Encoder:** A frozen *Sentence-Transformer* (`all-mpnet-base-v2`) generates a 768-dimensional embedding from the input text.
2. **Classifier:** A 3-layer feed-forward neural network:
 - `Linear(768, 512) --> BatchNorm --> GELU --> Dropout(0.4)`
 - `Linear(512, 256) --> BatchNorm --> GELU --> Dropout(0.3)`
 - `Linear(256, 96) --> Softmax Output`

The core model definition is (which takes encoder's embeddings as input):

```
self.model = nn.Sequential(  
    nn.Linear(input_dim, 512),  
    nn.BatchNorm1d(512),  
    nn.GELU(),  
    nn.Dropout(0.4),  
    nn.Linear(512, 256),  
    nn.BatchNorm1d(256),  
    nn.GELU(),  
    nn.Dropout(0.3),  
    nn.Linear(256, num_classes),  
)
```

6. Theoretical Analysis

The use of **Label Smoothing** (0.1) in the CrossEntropy loss is critical here because different emojis often share semantic space (e.g., various "heart" emojis). Label smoothing prevents the model from becoming overconfident in a single class, which explains the high Top-5 accuracy. The **OneCycleLR** policy is employed to allow for faster convergence and regularize the model by oscillating the learning rate.

7. Data Preprocessing and Training Methodology

- **Preprocessing:** Descriptions were "exploded" from the CSV, shuffled, and split into 80% training and 20% validation sets.
- **Caching:** To speed up training, text embeddings were pre-computed and cached as tensors.
- **Framework:** PyTorch Lightning was used for structured training, implementing *EarlyStopping* (patience=15) and *ModelCheckpointing* to prevent overfitting.

8. Experimental Results

The model was evaluated on the validation set, yielding the following results:

Overall result summary (we can see top-5 accuracy is much better than balanced accuracy, likely due to semantic similarities between some emojis):

Metric	Value
Balanced Accuracy	75.6%
Top-5 Accuracy	92.46%
Precision (Avg)	78%
Recall (Avg)	76%
F1-Score (Avg)	75%

Full metrics from *report.txt* :

=====				
SKLEARN CLASSIFICATION REPORT				
=====				
	precision	recall	f1-score	support
🌡️	0.77	1.00	0.87	10
☕	1.00	0.88	0.93	8
✈️	1.00	0.50	0.67	10
💡	0.89	0.80	0.84	10
💊	0.50	0.60	0.55	10
😊	0.58	0.70	0.64	10
⚽	0.78	0.70	0.74	10
🏠	0.78	0.70	0.74	10
🍰	0.57	0.80	0.67	10
🕒	0.89	0.80	0.84	10
🖼️	0.64	0.70	0.67	10
😍	0.89	0.80	0.84	10
🔑	0.70	0.70	0.70	10
📡	0.80	0.50	0.62	8
❄️	0.67	0.80	0.73	10
🏀	0.88	0.70	0.78	10
🎾	0.82	0.90	0.86	10
🙏	0.89	0.80	0.84	10
🔭	0.50	0.70	0.58	10
📏	0.86	0.67	0.75	9
❤️	1.00	0.80	0.89	10
📄	0.75	0.60	0.67	10
🕒	0.89	0.80	0.84	10
😞	0.78	0.70	0.74	10
🗺️	0.90	0.90	0.90	10
✍️	1.00	0.40	0.57	10
🚂	0.54	0.70	0.61	10
🌟	0.52	0.80	0.67	10

	0.55	0.90	0.67	10
🌞				
🐶	0.71	0.56	0.62	9
🍎	0.90	0.90	0.90	10
🍔	1.00	0.70	0.82	10
🔧	0.69	0.90	0.78	10
🔧	0.73	0.80	0.76	10
💰	0.83	1.00	0.91	10
📦	1.00	0.90	0.95	10
🚀	1.00	0.40	0.57	10
📷	1.00	1.00	1.00	10
😄	0.67	1.00	0.80	10
🌍	0.73	0.89	0.80	9
✨	0.89	0.80	0.84	10
🎈	0.78	0.70	0.74	10
🍷	0.80	0.80	0.80	10
🚲	0.88	0.78	0.82	9
🔨	0.55	0.67	0.60	9
🧼	0.89	0.80	0.84	10
🔬	0.50	0.75	0.60	8
🧹	0.80	0.89	0.84	9
🍺	0.82	0.90	0.86	10
😎	0.75	0.67	0.71	9
🔒	0.89	0.89	0.89	9
✂️	1.00	0.70	0.82	10
😬	0.77	1.00	0.87	10
🎈	1.00	0.90	0.95	10
📈	0.60	0.90	0.72	10
🧸	0.70	0.78	0.74	9
👍	0.40	0.44	0.42	9
🥗	0.55	0.67	0.60	9
💀	1.00	0.56	0.71	9
😭	1.00	0.50	0.67	10
🏠	1.00	0.78	0.88	9
😬	0.75	0.67	0.71	9
🚽	0.89	0.89	0.89	9
📁	0.78	0.78	0.78	9
🕯️	0.60	0.30	0.40	10
🌈	0.57	0.80	0.67	10
🏠	0.78	0.70	0.74	10
🕒	0.83	0.50	0.62	10
📱	0.67	1.00	0.80	10
🔧	0.64	0.70	0.67	10
📖	0.90	0.90	0.90	10
🔥	0.62	1.00	0.77	10
🍷	0.82	0.90	0.86	10
🧸	1.00	0.90	0.95	10
🧺	0.75	0.90	0.82	10
❤️	0.82	0.90	0.86	10
🚢	0.78	0.78	0.78	9
🧪	0.70	0.78	0.74	9
🍕	0.67	0.44	0.53	9
🧹	0.62	1.00	0.76	8
📦	0.67	0.89	0.76	9
💻	0.43	0.30	0.35	10
📚	1.00	0.80	0.89	10
🎈	0.82	0.90	0.86	10

	0.73	0.80	0.76	10
	0.50	0.50	0.50	10
	1.00	0.78	0.88	9
	0.80	0.89	0.84	9
	0.50	0.50	0.50	10
	0.78	0.78	0.78	9
	1.00	0.50	0.67	10
	1.00	0.78	0.88	9
	0.82	0.90	0.86	10
	0.89	0.80	0.84	10
	0.89	0.89	0.89	9
	1.00	0.70	0.82	10
	0.55	0.60	0.57	10
accuracy			0.76	927
macro avg	0.78	0.76	0.75	927
weighted avg	0.78	0.76	0.75	927
Balanced Accuracy:	0.7560			
Top-5 Accuracy:	0.9246			

Also full confusion matrix is saved at *evaluated/confusion_matrix.csv* but due to 96 output classes it's difficult to interpret and above metrics are more relevant to understand model.

9. Inferences / Insights Obtained / Difficulties faced

Challenges with Performance and Data Volume

The project faced significant initial hurdles regarding data scarcity:

- **Initial Performance:** The first iterations yielded very poor performance, starting at **7%** and only improving to **12%** accuracy.
- **Root Cause:** The bottleneck was identified as having too little training and validation data. Initially, there were only 10 descriptions per class across 100 emoji classes.
- **The Data Breakthrough:** The biggest performance gain came from increasing the dataset size. Using **Gemini LLM** to generate synthetic data, the dataset was expanded from 10 to **50 descriptions per emoji class** (across 96 classes). This shift alone increased the accuracy to **67%**.

Evolution of Embeddings

- **Custom Embeddings:** At the 67% accuracy mark, the model was still using `nn.EmbeddingBag()` with embeddings trained from scratch.
- **Transfer Learning:** Further gains were achieved by switching to pre-trained encoders. Moving to `SentenceTransformer('all-MiniLM-L6-v2')` boosted accuracy to **73%**.
- **Final Optimization:** Switching to the more robust `'all-mpnet-base-v2'` model provided the final push to **75.6%** accuracy.

General Insights

- **Semantic Overlap:** The gap between Top-1 and Top-5 accuracy indicates that "misclassifications" are often just "semantically similar" choices, which is acceptable for a search tool.
- **Efficiency:** The model's low parameter count makes inference near-instantaneous on a CPU, proving that full LLMs are overkill for this specific task.
- **Visual Complexity:** Analyzing a 96-class confusion matrix proved difficult; future work could involve grouping emojis into "Super-classes" (e.g., Faces, Food, Nature) for better error analysis.